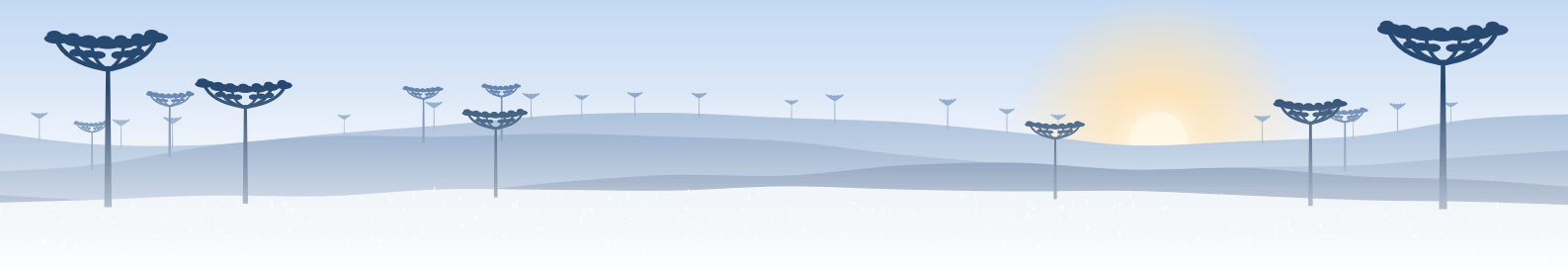
Guarapuava em números



APOSTILA

Como o portal foi feito, passo a passo

De onde vêm os dados, o que o código faz, quais bibliotecas foram usadas e por quê. Em linguagem simples, para quem está começando a programar.

Versão do portal	**1.2.4**
Data	**13 de novembro de 2026**
Portal	**guarapuava-em-numeros.streamlit.app**
Código	**github.com/marcelosido/portal-guarapuava**

Preparada para o Marcelo, com o Claude. Todos os números citados vêm dos dados reais do portal (coleta de 12/11/2026) e do código da versão 1.2.4.

Sumário

INTRODUÇÃO

01 Como usar esta apostila 3

PARTE 1 · O CAMINHO DOS DADOS

02 Como o portal funciona 4

03 As ferramentas e por que cada uma 5

04 O mapa do projeto 7

05 De onde vêm os dados (as APIs) 9

06 O robô de coleta 11

PARTE 2 · AS CONTAS

07 Dos relatórios aos indicadores 13

08 Compras públicas e LGPD 15

09 Comparação entre cidades 17

10 Pontos de atenção 19

PARTE 3 · O SITE

11 O site em Streamlit 21

12 Gráficos, cores e cabeçalho 24

PARTE 4 · QUALIDADE E MANUTENÇÃO

13 Testes automáticos 26

14 Publicar e manter 28

PARTE 5 · NA PRÁTICA

15 Estudo de caso: a versão 1.2.4, do pedido à entrega 29

16 Para praticar 32

17 Glossário 34

Cada seção começa numa página nova. Os trechos de código têm comentários linha a linha; o código completo está no GitHub, no arquivo indicado antes de cada trecho.

INTRODUÇÃO

01 Como usar esta apostila

Esta apostila explica, passo a passo, como o portal **Guarapuava em números** foi construído: de onde vêm os dados, como o robô os busca, que contas são feitas e como o site desenha tudo. Ela foi escrita para quem nunca programou. Não é preciso decorar nada: a ideia é entender a lógica e saber onde procurar.

O caminho da apostila

Seções	Assunto
2 a 6	O caminho dos dados: das fontes oficiais até a pasta `data/`
7 a 10	As contas: indicadores, compras, comparação entre cidades e pontos de atenção
11 e 12	O site: páginas, gráficos, cores e o cabeçalho
13 e 14	Testes automáticos, publicação e manutenção
15	Estudo de caso: a versão 1.2.4, do seu pedido até a entrega
16 e 17	Exercícios para praticar e glossário

Como ler os trechos de código

- Tudo o que vem depois de um `#` é um **comentário**: uma anotação para pessoas, que o computador ignora. Os comentários desta apostila explicam cada linha.
- Os trechos com `...` foram encurtados. O código completo está no GitHub, no arquivo indicado logo antes do trecho (por exemplo, `despesas/painel.py`).
- Os nomes no código estão em português de propósito: `receita`, `populacao`, `contratos`. Fica mais fácil de ler.

Duas convenções do projeto

Arquivos `f` e `d`. Os arquivos de dados seguem o padrão do Power BI: `f` de **fato** (os valores, como `fDespesa`) e `d` de **dimensão** (os nomes que explicam os códigos, como `dFuncao`).

Regra de ouro. Nenhum número é inventado ou estimado. Quando um dado não existe na fonte oficial, ele aparece como "–" ou "sem dado", e o portal explica o motivo.

Dica de estudo: deixe o repositório aberto no GitHub ao lado da apostila (github.com/marcelosido/portal-guarapuava) e abra cada arquivo citado. Ver o código inteiro, mesmo sem entender tudo, ajuda a perder o medo.

PARTE 1 · O CAMINHO DOS DADOS

02 Como o portal funciona

Pense numa banca de jornal. Toda segunda-feira um entregador (o **robô**) passa nas agências oficiais (as **APIs** do Tesouro Nacional e do PNCP), pega as notícias novas (os **dados**) e arruma tudo numa prateleira (a pasta `data/` no GitHub). A banca (o **site**, no Streamlit) mostra para quem passa o que está na prateleira.

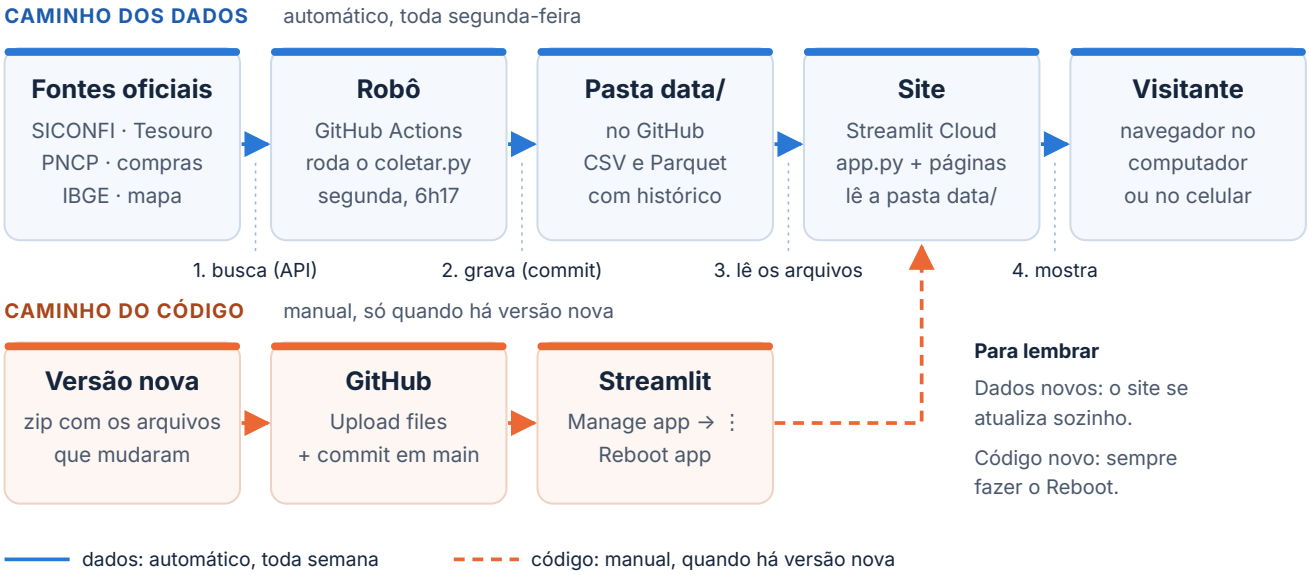


Figura 1. O caminho dos dados (em azul, automático) e o caminho do código (em laranja, manual).

As cinco peças

1. **As fontes oficiais.** O SICONFI (Tesouro Nacional) recebe os relatórios contábeis e fiscais da prefeitura; o PNCP recebe as licitações, dispensas e contratos. As duas têm APIs públicas e gratuitas.
2. **O robô.** Um programa (`coletar.py`) que roda no GitHub Actions toda segunda-feira às 6h17, sem precisar de nenhum computador ligado.
3. **A pasta `data/`.** Arquivos CSV e Parquet no próprio repositório, que fazem o papel de banco de dados.
4. **O site.** O Streamlit Community Cloud lê a pasta `data/` e desenha as páginas.
5. **O navegador** de quem visita o portal.

Dois caminhos diferentes

- **O caminho dos dados** é automático (robô → `data/` → site) e acontece toda semana.
- **O caminho do código** é manual: você recebe um zip, sobe os arquivos no GitHub e reinicia o app (Reboot). Acontece só quando o portal ganha uma versão nova.

Por que este desenho

- **Custo zero.** GitHub, GitHub Actions e Streamlit Community Cloud são gratuitos para projetos públicos.
- **Nada para manter.** Não há servidor nem banco de dados para atualizar ou pagar.
- **Tudo auditável.** O código e o histórico dos dados são públicos.
- **Falhas isoladas.** Se uma API sair do ar, o site continua com os últimos dados coletados.

03 As ferramentas e por que cada uma

Antes da lista, duas palavras que aparecem muito:

- **Biblioteca** é uma caixa de ferramentas pronta que o seu código chama quando precisa. Exemplo: o pandas sabe somar colunas; o portal só pede "some esta coluna".
- **Framework** é uma estrutura pronta que chama o seu código. Exemplo: no Streamlit você escreve o conteúdo de cada página, e ele cuida do servidor, do menu, dos cliques e da tela.

Ferramenta	O que é	Para que serve no portal	Por que esta
Python 3.12	Linguagem de programação	Tudo: coleta, contas e site	Fácil de ler; a mais usada para dados; gratuita
requests	Biblioteca	Pedir dados às APIs pela internet	Simples e muito testada
pandas	Biblioteca	Tabelas: filtrar, somar, juntar, pivotar	É o "Excel do Python"
pyarrow	Biblioteca	Ler e gravar arquivos Parquet	Arquivo compacto e rápido para a tabela grande
Streamlit	Framework	O site inteiro, escrito em Python	Sem HTML nem JavaScript; hospedagem gratuita
Plotly	Biblioteca	Barras, linhas e medidores	Gráficos interativos (passar o mouse mostra o valor)
Altair	Biblioteca	Mapa clicável e faixas de ranking	Monta gráficos em camadas; avisa o clique
numpy	Biblioteca	Contar dias úteis entre datas	Já vem junto com o pandas
pytest	Ferramenta	Testes automáticos	O padrão do Python
GitHub	Site	Guarda o código e os dados, com histórico	Gratuito para projetos públicos
GitHub Actions	Serviço	O robô semanal	Gratuito; roda na nuvem
Streamlit Community Cloud	Serviço	Hospeda o site	Gratuito; acompanha o GitHub sozinho
GitHub Codespaces	Serviço	Um computador na nuvem para testar	Não precisa instalar nada

A lista de compras (`requirements.txt`)

As bibliotecas ficam listadas num arquivo de texto. O robô e o Streamlit leem essa lista e instalam tudo antes de começar:

```
requests>=2.31        # conversar com as APIs
pandas>=2.2           # tabelas
pyarrow>=15           # arquivos Parquet
plotly>=6.0           # gráficos
altair>=5.4           # mapa e rankings
streamlit>=1.66,<2    # o site
```

`>=2.2` quer dizer "a versão 2.2 ou uma mais nova". No Streamlit, `,<2` acrescenta "mas não a versão 2", que pode mudar a forma de escrever as páginas.

Por que Parquet e CSV

A tabela da despesa detalhada tem mais de 200 mil linhas. Em **Parquet** ela ocupa pouco espaço e abre rápido. As tabelas menores ficam em **CSV** com ponto e vírgula e vírgula decimal, o padrão brasileiro: abrem direto no Excel e no Power BI.

04 O mapa do projeto

Cada arquivo tem uma função só. A regra principal da organização: a pasta `despesas/` é o **motor** (busca dados e faz contas) e a pasta `paginas/` é a **vitrine** (só monta a tela). O robô usa o motor sem nunca abrir o site.

```
portal-guarapuava/
├── app.py                    porta de entrada do site: menu e cabeçalho
├── coletar.py                roteiro do robô: busca os dados e grava em data/
├── conciliar_atende.py       confere a despesa com o portal da transparência (uso manual)
├── requirements.txt          lista de bibliotecas
├── pyproject.toml            ficha do projeto (nome e versão)
├── README.md                 manual curto do projeto
├── Dockerfile                receita para rodar num servidor próprio (opcional)
├── .github/workflows/
│   └── atualizar-dados.yml   agenda e passos do robô
├── .streamlit/
│   └── config.toml           cores e fontes do site (tema claro e escuro)
├── despesas/                 o motor
│   ├── config.py             código IBGE, CNPJ, endereços das APIs, tempos
│   ├── api.py                conversa com as APIs (pausas e novas tentativas)
│   ├── transformacao.py      MSC → tabela fDespesa; conferência com o RREO
│   ├── indicadores.py        acha os números no RREO e no RGF
│   ├── pncp.py               compras: janelas de tempo, órgãos e LGPD
│   ├── comparativo.py        os 399 municípios: coleta e indicadores
│   ├── alertas.py            regras dos pontos de atenção
│   ├── formatacao.py         números no padrão brasileiro
│   ├── dicionarios.py        nomes de funções, naturezas e meses
│   ├── painel.py             peças comuns das páginas: cores, gráficos, tabelas
│   └── recursos/             cabecalho.svg, cabecalho.css e malha_pr.geojson
├── paginas/                  uma página do site por arquivo
│   ├── indicadores.py        Visão geral
│   ├── receitas.py           Receitas
│   ├── despesa.py            Despesas
│   ├── compras.py            Compras
│   ├── comparar.py           Comparar cidades
│   └── sobre.py              Sobre os dados
├── tests/                    testes automáticos
└── data/                     os dados, gravados pelo robô
```

O que tem na pasta `data/`

Arquivo	O que guarda
`fDespesa.parquet`	Despesa detalhada mês a mês (MSC): órgão, função, natureza, fonte; dotação, empenhado, liquidado e pago
`dNatureza.csv`, `dFuncao.csv`, `dPoderOrgao.csv`	Os nomes de cada código da despesa
`fIndicadores.csv`	Os números do RREO e do RGF, por período
`fReceitaCorrente.csv`	Receita mês a mês por imposto e transferência (RREO Anexo 3)
`rreo_totais.csv`, `conciliacao_rreo.csv`	A conferência MSC × RREO
`fContratacoes.csv`, `fContratos.csv`	Compras públicas (PNCP), sem números de documento de pessoas
`dMunicipios.csv`, `fComparativo.csv`, `comparativo_controle.csv`	Comparação entre os municípios do Paraná
`metadados.json`	Data da coleta, o que já foi feito, o que falta e as falhas

Por que separar o motor da vitrine

- **Reaproveitamento.** A mesma função que calcula um indicador serve ao robô, ao site e aos testes.
- **Mudanças seguras.** Trocar a cor de um gráfico mexe só na vitrine; mudar uma regra de cálculo mexe só no motor.
- **Outra cidade.** Para fazer o portal de outro município, basta trocar o código IBGE e o CNPJ em `config.py`.

05 De onde vêm os dados (as APIs)

Uma **API** é um endereço na internet que devolve dados em vez de uma página para pessoas. Funciona como um balcão de informações: você faz a pergunta no formato combinado e recebe a resposta num formulário padronizado.

Passo 1: a pergunta é um endereço (URL)

Para pedir ao SICONFI o gasto com pessoal de Guarapuava no 2º quadrimestre de 2026, o robô monta este endereço:

```
https://apidatalake.tesouro.gov.br/ords/siconfi/tt/rgf
    ?an_exercicio=2026                 ano
    &in_periodicidade=Q                Q = quadrimestral
    &nr_periodo=2                      2º quadrimestre (maio a agosto)
    &co_tipo_demonstrativo=RGF         Relatório de Gestão Fiscal
    &no_anexo=RGF-Anexo%2001           Anexo 1 (despesa com pessoal); %20 é o espaço
    &co_esfera=M                       M = município
    &co_poder=E                        E = Executivo (a Prefeitura)
    &id_ente=4109401                   código IBGE de Guarapuava
```

O `?` separa o endereço das perguntas; cada `&` acrescenta mais uma. Dá para colar esse endereço (tudo numa linha só) no navegador e ver a resposta.

Passo 2: a resposta vem em JSON

A resposta é um texto organizado em **JSON**: chaves `{ }` agrupam os campos de um registro, colchetes `[ ]` fazem uma lista e cada campo tem o formato `"nome": valor`. Uma das linhas da resposta real (só alguns campos):

```
{
  "cod_conta": "ReceitaCorrenteLiquidaLimiteLegal",
  "conta": "RECEITA CORRENTE LIQUIDA - RCL (IV)",
  "coluna": "Valor",
  "valor": 929125228.65
}
```

Ou seja: a receita corrente líquida dos 12 meses até agosto de 2026 foi de R$ 929,1 milhões. O robô procura linhas como essa pelo `cod_conta` e pela `coluna` (seção 7).

Passo 3: respostas grandes vêm em partes (paginação)

Uma consulta da MSC pode ter milhares de linhas. O SICONFI entrega até 5.000 por vez e avisa com `"hasMore": true` que há mais; o robô pede de novo a partir do ponto em que parou (`offset`). O PNCP entrega 50 registros por página e informa o total de páginas.

As fontes do portal

Fonte	O que traz	Frequência	Onde aparece
SICONFI · MSC orçamentária	Despesa detalhada (órgão, função, natureza, fonte)	Mensal	Despesas
SICONFI · RREO, Anexos 1, 3 e 14	Receitas, despesas, mínimos de educação e saúde, RCL	A cada 2 meses	Visão geral, Receitas
SICONFI · RGF, Anexos 1 e 2	Gasto com pessoal, dívida e seus limites	A cada 4 meses	Visão geral
SICONFI · Balanço Anual (DCA) e relatórios dos 399 municípios	Gasto por área, investimentos, população	Anual	Comparar cidades
PNCP · contratações e contratos	Licitações, dispensas e contratos	Contínua	Compras
IBGE · malha municipal	Contorno de cada cidade (arquivo dentro do projeto)	Fixa	Mapa

Boas maneiras com os serviços públicos

- **Sem chave de acesso:** as duas APIs são abertas.
- **Uma pausa entre pedidos:** 1 segundo no SICONFI e meio segundo no PNCP e na comparação, para não sobrecarregar os servidores.
- **Identificação:** cada pedido diz quem está perguntando (`portal-guarapuava/1.2.4 (dados abertos)`).
- **Defasagem:** a prefeitura tem até 30 dias para enviar cada relatório. Por isso o portal fica cerca de um mês atrás do portal da transparência.

06 O robô de coleta

O robô tem duas partes: a **agenda**, que diz quando e em que ordem trabalhar (`.github/workflows/atualizar-dados.yml`), e o **roteiro**, que diz o que buscar (`coletar.py`).

Passo 1: a agenda (GitHub Actions)

O arquivo de agenda é escrito em **YAML**, um formato de configuração em que a indentação (os espaços no começo da linha) mostra o que está dentro de quê. Versão encurtada:

```yaml
name: Atualizar dados
on:
  schedule:
    - cron: "17 9 * * 1"          # toda segunda, 9h17 no horário UTC = 6h17 em Brasília
  workflow_dispatch:              # também roda pelo botão "Run workflow"
permissions:
  contents: write                 # licença para gravar os dados no repositório
jobs:
  coletar:
    runs-on: ubuntu-latest        # um computador Linux emprestado pelo GitHub
    timeout-minutes: 90           # se passar de 90 minutos, o GitHub interrompe
    steps:
      - uses: actions/checkout@v4             # baixa o projeto
      - uses: actions/setup-python@v5         # prepara o Python 3.12
      - run: pip install -r requirements.txt  # instala as bibliotecas
      - name: Coletar os dados na API do Tesouro
        continue-on-error: true               # uma falha não impede de gravar o resto
        run: python coletar.py
      - name: Gravar os dados no repositório
        run: |
          git add data/
          git commit -m "Atualiza dados do Tesouro (data de hoje)"   # só se algo mudou
          git push
```

Lendo o `cron`. São cinco campos: minuto, hora, dia do mês, mês e dia da semana. `17 9 * * 1` quer dizer "minuto 17, hora 9, qualquer dia, qualquer mês, segunda-feira (1)". O minuto 17 foi escolhido de propósito: muitos robôs no mundo começam na hora cheia, e nesse horário o GitHub costuma atrasar as execuções.

Passo 2: o roteiro (`coletar.py`)

O roteiro tem três **módulos**, sempre nesta ordem, e um relógio de 70 minutos para o trabalho todo:

```python
def main():
    prazo = Prazo(70)                          # relógio: 70 minutos de coleta
    if "municipio" in modulos:
        modulo_municipio(...)                  # MSC, RREO e RGF de Guarapuava (SICONFI)
    if "pncp" in modulos:
        coletar_pncp(cliente, ...)             # compras, do trimestre mais recente
    if "comparativo" in modulos:
        coletar_comparativo(cliente, ...)      # os 399 municípios do Paraná, aos poucos
    gravar("metadados.json", metadados)        # o que foi feito, o que falta e as falhas
    print("Resumo da coleta")                  # aparece no fim do log do GitHub
```

O que não couber nos 70 minutos continua na execução seguinte. Os 20 minutos de folga até o limite de 90 servem para instalar as bibliotecas e gravar os dados.

Passo 3: buscar só o que mudou (coleta incremental)

Buscar tudo desde 2019 toda semana seria lento e desnecessário. O robô compara o que já tem com o que existe na API e busca:

- os meses e períodos que ainda não tem;
- os **3 últimos meses** da MSC e os **2 últimos** bimestres e quadrimestres de novo, porque a prefeitura pode corrigir (retificar) um envio.

Passo 4: pedir com educação e insistir sem exagero (`despesas/api.py`)

A função `_get` faz cada pedido. Versão simplificada:

```
for tentativa in range(1, 6):                       # até 5 tentativas
    esperar_a_pausa()                               # 1 s desde o último pedido
    resposta = sessao.get(url, timeout=90)          # pede; desiste da tentativa após 90 s
    if resposta.status_code == 200:                 # 200 = deu certo
        return resposta.json()
    if resposta.status_code == 204:                 # 204 = deu certo, mas não há nada
        return None
    if resposta.status_code in {429, 500, 502, 503, 504}:   # problema passageiro do servidor
        time.sleep(min(60, 2 ** tentativa))         # espera 2, 4, 8, 16 s e tenta de novo
    else:
        raise ErroAPI(...)                          # erro definitivo: não adianta insistir
```

Os números são **códigos de status** do protocolo HTTP, a língua dos sites: 200 é "ok", 404 é "não existe", 429 é "muitos pedidos, espere um pouco" e os da casa dos 500 são "o servidor está com problema". A espera que dobra a cada tentativa (2, 4, 8, 16 segundos) dá tempo para o servidor se recuperar.

Passo 5: gravar sem nunca deixar um arquivo pela metade

```
def _gravar(caminho, escrever):
    temporario = caminho.with_name(caminho.name + ".tmp")   # fIndicadores.csv.tmp
    escrever(temporario)                                     # grava tudo no temporário
    os.replace(temporario, caminho)                          # troca de uma vez só
```

Se o robô parar no meio da gravação, o arquivo antigo continua inteiro. O site nunca lê um arquivo incompleto.

Passo 6: o resumo no fim do log

No fim de cada execução aparece o **Resumo da coleta**. Trecho da execução de 12/11/2026:

```
Resumo da coleta
  MSC: 01/2019 a 09/2026 (93 meses, 204.076 linhas)
  ...
  RREO até o 4º bimestre/2026
  RGF até o 2º quadrimestre/2026
  ...
  PNCP: 977 contratações e 1061 contratos (ok, 150 requisições)
  Comparativo PR 2025: 394 de 399 municípios com dados (parcial, 0 requisições)
    O restante entra nas próximas execuções do robô.
```

"Parcial" com 0 requisições quer dizer que não havia nada novo para buscar: 12 municípios ainda não entregaram algum relatório de 2025 (5 deles, nenhum), e o robô só volta a procurar depois de 7 dias (seção 9).

07 Dos relatórios aos indicadores

Cada número do portal já existe num relatório oficial: o código só precisa achá-lo no meio de centenas de linhas. Ele procura por duas pistas, o **código da conta** (`cod_conta`) e o **nome da coluna** (`coluna`), e nunca pela posição da linha, que muda de um ano para outro.

Passo 1: a lista de regras (`despesas/indicadores.py`)

Para cada indicador há uma regra com três partes: o nome que o portal usa, o começo do código da conta e a coluna. Trecho real das regras do RREO Anexo 14:

```
REGRAS_RREO14 = [
    # (nome no portal, começo do cod_conta, coluna)
    ("receita_realizada", "ReceitasRealizadasDemonstrativoSimplificado", r"^ate o bimestre"),
    ("rcl", "ReceitaCorrenteLiquidaDemonstrativoSimplificado", r"^ate o bimestre"),
    ("educacao_pct",
        "MinimoAnualDasReceitasDeImpostosNaManutencaoEDesenvolvimentoDoEnsino", r"^% aplicado"),
    ("saude_pct",
        "AplicacaoTotalDasDespesasComAcoesEServicosPublicosDeSaude", r"^% aplicado"),
    ...
]
```

O `r"^ate o bimestre"` é uma **expressão regular** (regex): um molde de texto. O `^` quer dizer "começa com". Assim a regra aceita "Até o Bimestre", "Até o Bimestre (c)" e outras variações.

Passo 2: a função que procura (`_primeiro`)

```
def _primeiro(itens, prefixo, coluna):
    regex = re.compile(coluna)                          # prepara o molde da coluna
    for item in itens:                                  # percorre as linhas do relatório
        if (item["cod_conta"].startswith(prefixo)       # o código começa como esperado?
                and regex.search(normalizar(item["coluna"]))):   # e a coluna bate?
            return round(float(item["valor"]), 2)       # achou: devolve o valor
    return None                                         # não achou: fica em branco
```

A função `normalizar` deixa o texto em minúsculas e sem acentos ("Até o Bimestre" vira "ate o bimestre"). Assim uma diferença de acento ou de maiúscula não impede de achar o número.

> **Regra de ouro do projeto:** quando um código não aparece, o indicador fica em branco. Nada é estimado ou inventado.

Passo 3: os limites também vêm do relatório

Os limites de gasto com pessoal (alerta, prudencial e máximo) não estão escritos no código. Eles são lidos do próprio RGF que a prefeitura envia, pelas contas `LimiteDeAlertaDespesaComPessoalTotal`, `LimitePrudencialDespesaComPessoalTotal` e `LimiteMaximoDespesaComPessoalTotal`.

Passo 4: a despesa detalhada (`despesas/transformacao.py`)

A MSC mensal vira a tabela `fDespesa`: uma linha por combinação de órgão, função, subfunção, tipo de gasto (natureza) e fonte do dinheiro, com as colunas dotação, empenhado, liquidado e pago. Os códigos viram nomes por meio de tabelas auxiliares: `dNatureza`, `dFuncao` e `dPoderOrgao`.

Esse desenho se chama **modelo estrela**: uma tabela de fatos (`f`, os valores) e tabelas de dimensões (`d`, os nomes). É o mesmo padrão usado no Power BI. A tabela de fatos é gravada em Parquet porque tem milhões de células.

Passo 5: a conferência

Para ter certeza de que nada se perdeu no caminho, o código soma a MSC e compara com o total que o RREO informa. A página "Sobre os dados" mostra cada diferença, bimestre a bimestre.

PARTE 2 · AS CONTAS

08 Compras públicas e LGPD

As compras vêm do PNCP em duas consultas: as **contratações** (o processo publicado) e os **contratos** (quem foi contratado). O código busca as duas por trimestre, do mais recente para o mais antigo, e protege os dados de pessoas físicas antes de gravar.

Passo 1: dividir o tempo em janelas (`despesas/pncp.py`)

A API do PNCP não aceita períodos longos. Por isso o código divide o tempo desde janeiro de 2024 em **trimestres** (as "janelas") e consulta um por vez. Se a API recusar até o trimestre, o código tenta mês a mês.

```
todas = janelas(inicio, hoje)          # 2024T1, 2024T2, ..., 2026T4
ordem = todas[::-1]                    # [::-1] inverte a lista: o mais recente primeiro

for janela in ordem:
    buscar_contratacoes(janela)        # todas as modalidades daquele trimestre
    contratos_da_janela(janela)        # depois os contratos de cada órgão, Prefeitura primeiro
```

Por que nessa ordem? Se o tempo do robô acabar no meio, o que fica faltando é sempre o período mais antigo. A primeira coleta da versão 1.2 ensinou isso: o tempo acabou antes dos contratos da Prefeitura.

Passo 2: ficar só com a prefeitura e seus órgãos

A consulta traz tudo o que foi publicado com o código IBGE de Guarapuava, inclusive órgãos estaduais e federais (uma universidade, por exemplo). O código fica só com a esfera municipal:

```
linhas = [x for x in linhas
          if x["esfera"] == "M"                              # esfera municipal
          and x["municipio_ibge"] in ("", "4109401")]        # e de Guarapuava
```

Essa forma `[x for x in lista if condição]` é uma **compreensão de lista**: "monte uma lista nova só com os itens que passam no teste".

Passo 3: lembrar o que já foi feito

Cada trimestre coletado é anotado no `metadados.json` (`janelas_ok`, `contratos_ok`). Na execução seguinte, o robô pula o que já tem e refaz só os dois trimestres mais recentes, que ainda recebem atualizações.

Passo 4: proteger os dados pessoais (LGPD)

Antes de gravar, cada fornecedor passa pela função `fornecedor_publico`:

```
def fornecedor_publico(tipo, documento, nome):
    if tipo == "PF" or len(documento) == 11:          # pessoa física (CPF tem 11 dígitos)
        return "PF", "", _sem_documento(nome)         # nome sim, documento não
    if _MEI.search(nome) or _MEI_CNPJ.match(nome):    # MEI: número de documento no nome
        return "MEI", "", _sem_documento(nome)        # nome sem o número, sem CNPJ
    return tipo or "PJ", documento, nome              # empresa: CNPJ e nome são públicos
```

`_sem_documento` apaga qualquer CPF que apareça no nome. Em textos livres, como o objeto da compra, a função `mascarar_cpf` troca números com formato de CPF por `***.***.***-**`. Ela usa uma regex que reconhece "três dígitos, ponto, três dígitos, ponto, três dígitos, traço, dois dígitos", com ou sem pontuação.

No site, os nomes de pessoas físicas e MEIs aparecem **só nas tabelas**. Nos rankings e nos pontos de atenção eles ficam somados num grupo só.

Passo 5: quem foi contratado (a coluna "Contratado")

Cada contrato no PNCP traz o número de controle da contratação de origem. A função `contratados_por_compra` usa esse número para ligar os dois:

```
k = contratos[contratos["id_compra"] != ""]         # só contratos ligados a uma compra
k = k.sort_values("valor_global", ascending=False)   # o de maior valor primeiro
nomes = k.groupby("id_compra")["fornecedor_nome"].agg(juntar)
# juntar: "EMPRESA A" ou, com várias empresas, "EMPRESA A e mais 2"
```

O `groupby` agrupa as linhas pelo número da compra, como uma tabela dinâmica do Excel. Uma licitação sem contrato publicado ainda fica com "–".

09 Comparação entre cidades

A página Comparar usa os relatórios dos 399 municípios do Paraná, coletados aos poucos pelo robô. Desde a versão 1.2.4, a medida principal de cada área é o **peso no orçamento**, e a comparação pode ser com todo o estado ou só com **cidades do mesmo porte**.

Passo 1: a coleta em etapas, que pode ser retomada (`despesas/comparativo.py`)

Para cada município há cinco consultas: o extrato de entregas, o RREO Anexo 14, o RGF Anexo 1 e dois anexos do Balanço Anual (DCA I-D e I-E). São cerca de 2.000 consultas, mais do que cabe numa execução.

Por isso cada resultado é anotado no arquivo `comparativo_controle.csv`, com a situação: `ok`, `nao_entregue` (o município ainda não enviou o relatório) ou `falha`. Na execução seguinte o robô pula o que está `ok` e continua o resto. Um relatório não entregue é procurado de novo depois de 7 dias.

Passo 2: de linhas para uma tabela (`tabela_indicadores`)

Os dados chegam "em pé": uma linha por município e por número (`cod_ibge`, `chave`, `valor`). A função vira a tabela "deitada", com uma linha por município e uma coluna por indicador, como uma tabela dinâmica:

```
largo = dados.pivot_table(index="cod_ibge", columns="chave", values="valor", aggfunc="first")
```

Depois calcula as medidas. As duas principais:

```
# gasto por habitante: despesa liquidada na área ÷ população
base["educacao_hab"] = base["funcao_12"] / populacao

# peso no orçamento: despesa na área ÷ despesa total, em %
base["educacao_peso"] = base["funcao_12"] / base["despesa_total"] * 100
```

`funcao_12` é a despesa liquidada na função 12 (Educação) do Balanço Anual, sem as transferências entre órgãos da própria prefeitura. Saúde é a função 10, transporte a 26, urbanismo a 15 e assistência a 08.

Passo 3: por que o peso no orçamento é a medida principal

Cidades pequenas quase sempre gastam mais por habitante: têm custos fixos e recebem mais repasses por pessoa. No gasto por habitante, Guarapuava fica em 368º de 388 em educação, o que dá a impressão de gastar pouco. No peso no orçamento, 26,4% do gasto vai para a educação, o que a coloca em 92º no estado e em 10º entre as 21 cidades do mesmo porte.

O gasto por habitante continua na página, embaixo de cada faixa, como dado de apoio.

Passo 4: cidades do mesmo porte

As cidades são separadas em faixas de população: até 20 mil, de 20 mil a 50 mil, de 50 mil a 100 mil, de 100 mil a 900 mil e mais de 900 mil habitantes. "Mesmo porte" é a mesma faixa.

```
FAIXAS_PORTE = [
    (20_000, "com até 20 mil habitantes"),          # (limite, nome da faixa)
    (50_000, "de 20 mil a 50 mil habitantes"),
    (100_000, "de 50 mil a 100 mil habitantes"),
    (900_000, "de 100 mil a 900 mil habitantes"),
    (float("inf"), "com mais de 900 mil habitantes"),   # inf = infinito: sem limite
]
```

Uma faixa com menos de 8 cidades (só Curitiba passa de 900 mil) usa as 12 cidades de população mais próxima. "Mais próxima" é medida em proporção, com logaritmo: a distância de 100 mil para 200 mil é a mesma de 1 milhão para 2 milhões.

Passo 5: a posição no ranking

```
posicao = serie.rank(ascending=False, method="min")
```

`rank` numera do maior para o menor (`ascending=False`). Com `method="min"`, cidades empatadas ficam na mesma posição. A faixa colorida divide as cidades em cinco partes iguais (quintis), nas mesmas cores do mapa.

Passo 6: o mapa que dá para clicar

O mapa usa a malha oficial dos municípios (um arquivo GeoJSON, que guarda o contorno de cada cidade como uma lista de coordenadas) e é desenhado pelo Altair. Um clique numa cidade guarda o código dela na memória da página (`st.session_state`), e o seletor "Comparar com" passa a mostrá-la. No modo "mesmo porte", as cidades de fora do grupo aparecem em cinza claro.

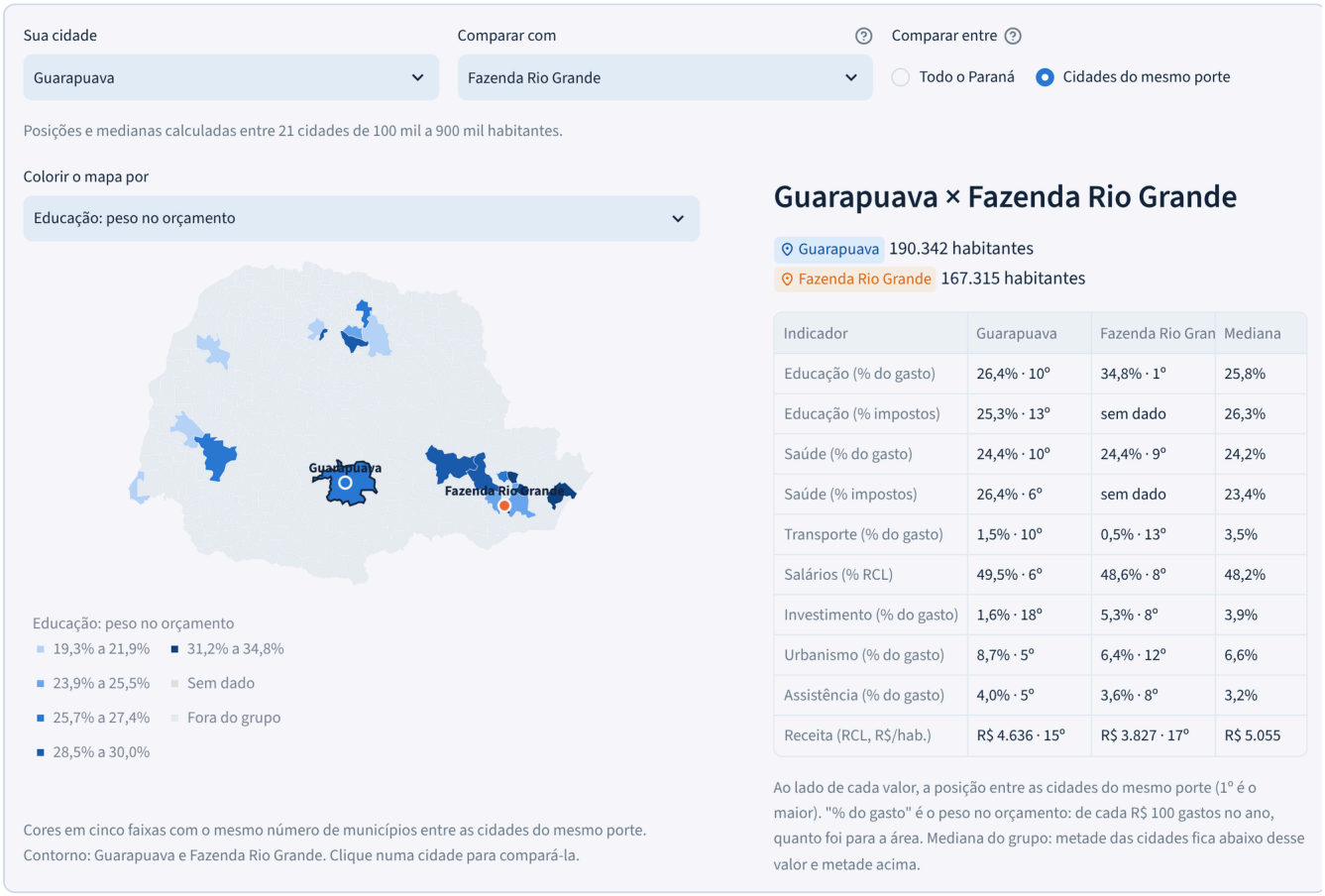


Figura 2. A página Comparar na versão 1.2.4, entre as 21 cidades de 100 mil a 900 mil habitantes (dados reais de 2025). Ao lado de cada valor, a posição no grupo.

10 Pontos de atenção

Os pontos de atenção são regras fixas, escritas no arquivo `despesas/alertas.py`, que comparam cada número com um limite da lei ou com um critério do portal. Eles não acusam ninguém: indicam o que vale perguntar e conferir na fonte.



Figura 3. Os pontos de atenção na Visão geral, com os dados até agosto de 2026. Cada selo tem cor, ícone e texto.

Passo 1: o formato de um ponto

Cada ponto é um pequeno registro com os mesmos campos. Em Python isso se escreve com uma **dataclass**, um molde de ficha:

```python
@dataclass
class Ponto:
    nivel: str                    # "critico", "grave", "alerta", "info" ou "incompleto"
    area: str                     # "Contas públicas" ou "Compras"
    titulo: str                   # a frase em negrito
    detalhe: str                  # a explicação com os números
    pagina: str | None = None     # onde ver mais
    quantidade: int | None = None # quantos casos, quando é uma contagem
    regra: str = ""               # de onde vem o critério
    chave: str = ""               # um apelido, usado para abrir a lista certa
```

O `= None` e o `= ""` são valores padrão: os campos que vêm depois do `detalhe` podem ficar de fora quando não se aplicam.

Os pontos são ordenados pela gravidade, e o nível vira um selo com **cor, ícone e texto**. A cor nunca aparece sozinha, para quem não enxerga cores.

Passo 2: um exemplo de regra (gasto com pessoal)

```
def status_limite(valor, alerta, prudencial, maximo):
    if valor is None or maximo is None:          # falta o dado: não arrisca um palpite
        return "sem_dado", "Sem dado"
    if valor > maximo:
        return "critico", "Acima do limite máximo"
    if valor > prudencial:
        return "grave", "Acima do limite prudencial"
    if valor > alerta:
        return "alerta", "Acima do limite de alerta"
    return "ok", "Dentro do limite"
```

A ordem dos testes importa. Quem passou do máximo também passou do prudencial, então o teste mais grave vem primeiro. Com os dados até agosto de 2026, a Prefeitura gastou 51,07% da receita corrente líquida com pessoal: acima do alerta (48,6%) e abaixo do prudencial (51,3%). Resultado: "alerta".

Passo 3: as regras e de onde vêm

Ponto	Critério	Base
Gasto com pessoal (Prefeitura)	Alerta 48,6%, prudencial 51,3%, máximo 54% da RCL	LRF, arts. 20, 22 e 59 (lidos do RGF)
Dívida consolidada líquida	Máximo 120% da RCL; alerta em 108%	Resolução do Senado 40/2001 e LRF, art. 59
Educação	Mínimo de 25% dos impostos	Constituição, art. 212
FUNDEB nos salários	Mínimo de 70%	Lei 14.113/2020, art. 26
Saúde	Mínimo de 15% dos impostos	LC 141/2012, art. 7º
Resultado primário	Meta da LDO do município	LDO
Receitas de capital e investimentos	Menos de 25% executado com 2/3 do ano passado	Critério do portal
Contratações emergenciais	Amparo no art. 75, VIII	Lei 14.133/2021
Várias dispensas com a mesma empresa	3 ou mais dispensas por valor no ano	Lei 14.133/2021, art. 75, § 1º
Prazo curto para propostas	Menos de 8 dias úteis	Lei 14.133/2021, art. 55
Licitação sem desconto	Homologado a menos de 1% do estimado	Critério do portal
Concentração de fornecedores	5 maiores com 50% ou mais dos contratos	Critério do portal

Passo 4: contar dias úteis

O prazo para propostas é medido em dias úteis. O código usa uma função pronta da biblioteca **numpy**, que conta os dias de segunda a sexta entre duas datas (`busday_count`). Feriados não são descontados, e a página avisa isso.

Passo 5: quando o dado está incompleto, a regra espera

Se a coleta dos contratos de algum órgão ainda não terminou, as regras que dependem de fornecedores ficam suspensas. No lugar delas aparece um ponto cinza "Coleta incompleta". Assim o portal nunca mostra um retrato pela metade como se fosse completo.

PARTE 3 · O SITE

11 O site em Streamlit

O site é feito com o **Streamlit**, que tem uma regra simples: a cada clique do visitante, o script da página roda de novo, de cima a baixo, e o Streamlit redesenha só o que mudou. Não é preciso escrever HTML nem JavaScript.

Passo 1: o menor exemplo possível

```
import streamlit as st

ano = st.selectbox("Ano", [2026, 2025])     # desenha um seletor e devolve o valor escolhido
st.metric("Ano escolhido", ano)             # desenha um cartão com o número
```

Quando a pessoa troca o ano, o script roda de novo, `ano` recebe o valor novo e o cartão muda. Todo o portal é feito desse jeito, só que com mais peças.

Passo 2: a porta de entrada (`app.py`)

```
# título que aparece na aba do navegador; "wide" = página larga
st.set_page_config(page_title="Guarapuava em números", layout="wide")

paginas = [
    st.Page("paginas/indicadores.py", title="Visão geral", default=True),
    st.Page("paginas/receitas.py", title="Receitas", url_path="receitas"),
    ...                                                          # as outras 4 páginas
]
pagina = st.navigation(paginas, position="top")   # menu no topo
topo_portal("Guarapuava em números", "...")       # cabeçalho com a ilustração, em todas as páginas
pagina.run()                                      # roda a página escolhida
```

Passo 3: a anatomia de uma página

Todas as páginas seguem a mesma ordem, o que ajuda o visitante a se achar. Os números abaixo são os da figura:

1. **Menu**, igual em todas as páginas (`st.navigation`, em `app.py`).
2. **Cabeçalho ilustrado** (`topo_portal`).
3. **Título, uma frase sobre a página e o seletor de ano** (`cabecalho` e `seletor_ano`).
4. **Fonte e data:** de onde vêm os números e quando foram atualizados (`st.caption`).
5. **Cartões** com os números principais (`st.metric`).
6. **Pontos de atenção**, quando a página tem.
7. **Caixas que abrem** com a lista completa (`st.expander`).

Mais abaixo vêm os gráficos e as tabelas (em algumas páginas, separados em abas, com `st.tabs`) e, no fim da maioria das páginas, a caixa "Como ler estes números".

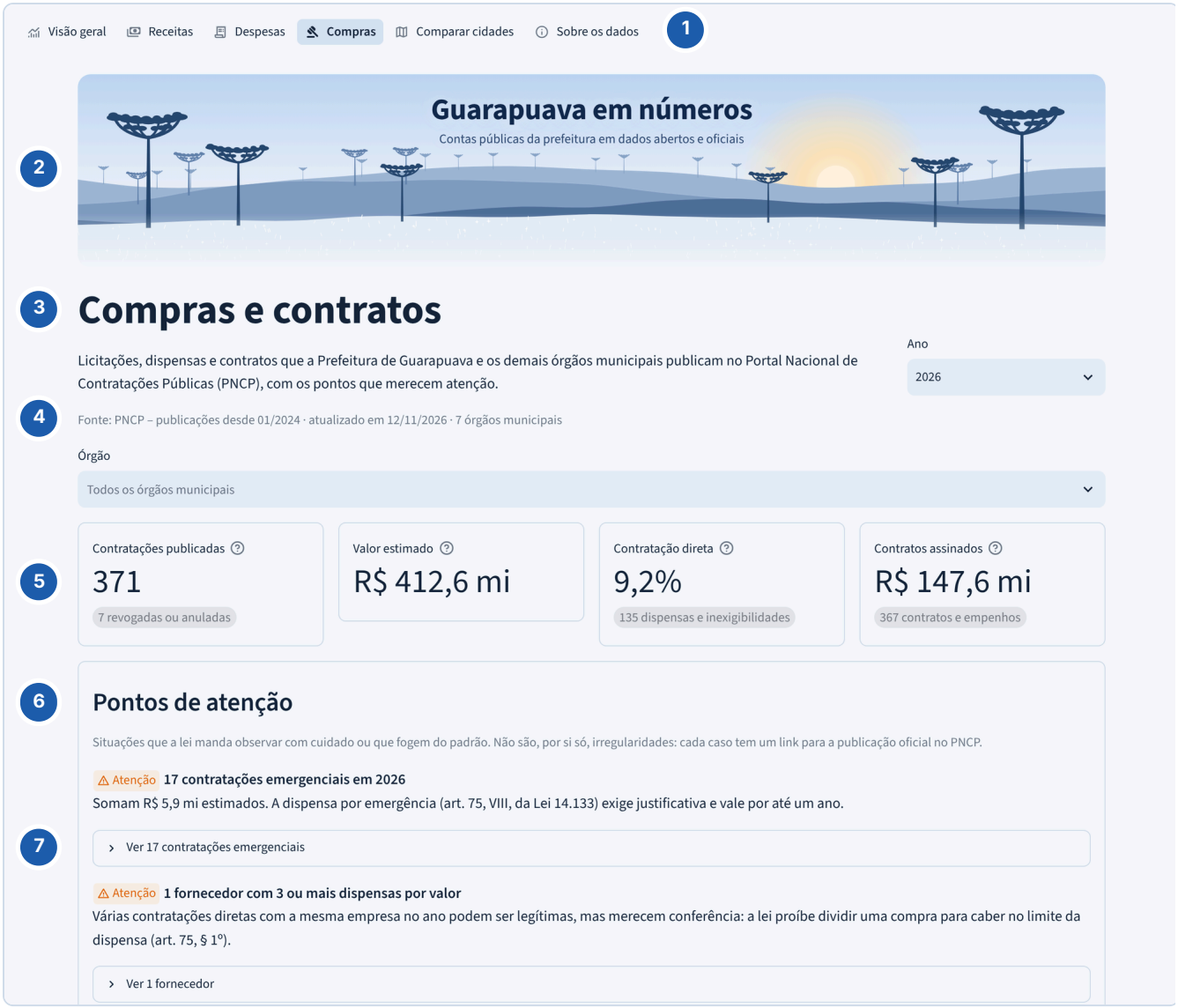


Figura 4. A página Compras, com as partes numeradas como na lista acima (dados reais, 12/11/2026).

Passo 4: a memória do visitante (`st.session_state`)

Como o script roda do zero a cada clique, as escolhas precisam ficar guardadas em algum lugar. O `st.session_state` é um caderninho por visitante. O ano escolhido numa página vale nas outras:

```
preferido = st.session_state.get("ano_escolhido")      # lê o ano escolhido antes
ano = coluna.selectbox(rotulo, anos, index=...)        # mostra o seletor já nesse ano
st.session_state["ano_escolhido"] = ano                # anota a nova escolha
```

Passo 5: não ler os arquivos a cada clique (cache)

Ler a tabela da despesa a cada clique deixaria o site lento. O `@st.cache_data` guarda o resultado da primeira leitura e o reaproveita:

```
@st.cache_data
def carregar_indicadores(pasta, versao):
    return pd.read_csv(Path(pasta) / "fIndicadores.csv", sep=";", decimal=",")

carregar_indicadores(str(PASTA), assinatura(PASTA / "fIndicadores.csv"))
```

O truque está no segundo argumento. `assinatura` devolve a data de modificação do arquivo. Quando o robô grava dados novos, a data muda, o cache entende que é outra leitura e carrega o arquivo novo sozinho.

Passo 6: as peças compartilhadas (`despesas/painel.py`)

Tudo o que se repete entre páginas mora em `painel.py`: o cabeçalho, o seletor de ano, os selos, a função `tabela` (números no padrão brasileiro e "–" no lugar de vazio) e os gráficos. Uma melhoria feita ali aparece em todas as páginas de uma vez.

PARTE 3 · O SITE

12 Gráficos, cores e cabeçalho

O portal usa duas bibliotecas de gráficos: o **Plotly** para barras e medidores, e o **Altair** para o mapa e as faixas de ranking. As cores seguem uma paleta testada para daltonismo e mudam sozinhas entre o tema claro e o escuro.

Passo 1: um gráfico de barras com Plotly

```
fig = go.Figure()
fig.add_bar(y=nomes, x=valores, orientation="h",          # barras deitadas: nomes longos cabem
            marker=dict(color=AZUL, cornerradius=4),      # cor da série e cantos arredondados
            text=[brl_compacto(v) for v in valores])      # o valor escrito na ponta da barra
estilo_grafico(fig, altura=300)                           # margens, fonte e legenda do portal
mostrar(fig, alt="Receita por fonte em 2026")             # desenha; "alt" descreve o gráfico em texto
```

As funções `estilo_grafico` e `mostrar` (em `painel.py`) garantem que todos os gráficos tenham o mesmo jeito. Dentro de `estilo_grafico`, a linha `separators=",."` faz o Plotly escrever os números no padrão brasileiro (1.234,5). O `theme="streamlit"` dentro de `mostrar` faz o fundo e as linhas seguirem o tema da página.

O medidor (previsto × realizado) são duas barras sobrepostas: uma clara, mais comprida, para o total previsto, e uma azul por cima, para o que já foi realizado.

Passo 2: por que o Altair no mapa

O Altair desenha o contorno dos municípios direto do arquivo do projeto (`malha_pr.geojson`), sem depender de mapas de fundo da internet, e o Streamlit avisa em qual cidade a pessoa **clicou** (`on_select="rerun"`). As faixas de ranking também são Altair: retângulos coloridos (as cinco partes), bolinhas (as duas cidades) e textos (nome e posição), empilhados em camadas, como folhas transparentes uma sobre a outra.

Passo 3: as cores

Cada tema tem seu conjunto de cores, chamado **tokens**, em `painel.py`:

```
TOKENS = {
    "light": {"serie": ["#2a78d6", "#eb6834", "#1baf7a"],     # azul, laranja, verde-água
              "superficie": "#f5f8fc", "tinta": "#15253b", ...},
    "dark":  {"serie": ["#3987e5", "#d95926", "#199e70"],
              "superficie": "#0d1626", "tinta": "#e7edf5", ...},
}
```

A função `tema()` pergunta ao Streamlit qual tema o visitante está usando e devolve as cores certas. Três regras valem em todos os gráficos:

- **Azul, laranja e verde-água**, sempre nessa ordem. A combinação foi conferida com um programa que simula os tipos mais comuns de daltonismo.
- **A cor nunca está sozinha:** todo selo tem cor, ícone e texto; todo gráfico tem legenda ou rótulo e uma tabela com os mesmos números.
- **Os textos usam a cor de texto**, e não a cor da série, para ter contraste suficiente.

Passo 4: o cabeçalho que vira noite

O desenho do topo é um arquivo **SVG** (`despesas/recursos/cabecalho.svg`): uma imagem feita de formas descritas em texto, como "um círculo aqui, uma curva ali". Ele foi gerado por um programa que desenha as colinas, as araucárias, o sol e a geada.



Figura 5. O mesmo arquivo SVG nos dois temas. Só as cores mudam, pelas regras do `cabecalho.css` .

Cada forma tem uma "classe", e o arquivo `cabecalho.css` diz a cor de cada classe nos dois temas, com a função `light-dark(cor do dia, cor da noite)` :

```
.gp-k1 { stop-color: #bcd5f2; stop-color: light-dark(#bcd5f2, #132a52) }  /* topo do céu */
.gp-t3 { fill: #27486f; fill: light-dark(#27486f, #08111f) }   /* araucárias da frente */
```

A primeira declaração é a cor do dia, para navegadores antigos; a segunda escolhe sozinha entre dia e noite. As estrelas são transparentes de dia e brancas à noite.

> **Uma lição de depuração:** a primeira versão usou `st.html` e o desenho não aparecia. O `st.html` limpa o HTML por segurança e remove imagens SVG. A solução foi usar `st.markdown(..., unsafe_allow_html=True)` , que mantém o desenho.

Passo 5: as cores do site (`.streamlit/config.toml`)

O fundo azul-geada, o texto azul-marinho e as cores dos botões estão neste arquivo, separados para os temas claro e escuro. Como as mesmas cores aparecem nos gráficos, um teste automático confere se o `config.toml` e os `TOKENS` continuam iguais.

13 Testes automáticos

Um **teste automático** é um pequeno programa que usa o código do portal com dados conhecidos e confere se o resultado é o esperado. O projeto tem 92 testes, que rodam em cerca de 40 segundos e são executados antes de cada entrega.

Passo 1: como é um teste

Um teste da versão 1.2.4, de `tests/test_comparativo.py` (simplificado):

```
def test_peso_no_orcamento():
    """De cada R$ 100 gastos, quanto foi para a área."""
    dados = pd.DataFrame([                                    # dois municípios inventados
        {..., "cod_ibge": "1", "chave": "despesa_total", "valor": 1_000_000.0},  # gasto total
        {..., "cod_ibge": "1", "chave": "funcao_12", "valor": 264_000.0},        # educação
        {..., "cod_ibge": "2", "chave": "funcao_12", "valor": 5_000.0},          # "2" sem total
    ])
    t = comparativo.tabela_indicadores(dados, lista, 2025).set_index("cod_ibge")
    assert round(t.loc["1", "educacao_peso"], 6) == 26.4     # 264 mil de 1 milhão = 26,4%
    assert pd.isna(t.loc["2", "educacao_peso"])              # sem o total, fica em branco
```

A palavra `assert` quer dizer "garanta que". Se a conta estiver errada, o teste falha e aponta a linha. O **pytest** é a ferramenta que acha todos os testes do projeto e roda um por um.

> **Por que o `round`?** O computador guarda números com casas decimais de forma aproximada. Para ele, 264.000 ÷ 1.000.000 × 100 dá 26,400000000000002. Por isso o teste arredonda antes de comparar, e o portal arredonda antes de mostrar.

Passo 2: APIs de mentira, com linhas de verdade

Os testes não podem depender da internet: a API pode estar fora do ar, e os dados mudam toda semana. Por isso o projeto tem duas peças:

- `tests/fixtures_api.py`: linhas copiadas das respostas reais das APIs, com os códigos e nomes exatos. Servem de gabarito.
- `tests/servidor_simulado.py`: um pequeno servidor que roda no próprio computador e responde como o SICONFI e o PNCP. Ele também simula problemas: falhas, recusas, prazos curtos.

Assim dá para testar, por exemplo, o que acontece quando o tempo do robô acaba no meio da coleta, sem esperar 70 minutos.

Passo 3: testar as páginas como um visitante

O Streamlit tem o `AppTest`, que abre as páginas sem navegador, clica nos seletores e lê o que apareceu. Exemplo (simplificado) de `tests/test_app.py`:

```
at = abrir(pasta, monkeypatch, "paginas/comparar.py")         # abre a página
next(r for r in at.radio if r.label == "Comparar entre") \
    .set_value("Cidades do mesmo porte").run()                 # escolhe "mesmo porte"
assert not at.exception                                        # nenhum erro na tela
assert any("mesmo porte" in s.value for s in at.subheader)     # o título mudou
```

Passo 4: o que os testes conferem

Arquivo	Alguns exemplos do que é conferido
`test_indicadores.py`	Os números achados no RREO e no RGF batem com os relatórios reais de 2026
`test_transformacao.py`	A MSC vira a tabela da despesa sem perder valores
`test_coleta.py`	A coleta incremental busca só o que falta, e o fim do tempo não conta como falha
`test_pncp.py`	Nenhum CPF chega aos arquivos; o trimestre mais recente e a Prefeitura vêm primeiro
`test_comparativo.py`	Peso no orçamento, gasto por habitante e grupos do mesmo porte
`test_alertas.py`	Cada ponto de atenção aparece no caso certo, com a frase no singular quando é um só
`test_cabecalho.py`	O cabeçalho tem cor de dia e de noite e usa o mesmo fundo dos gráficos
`test_app.py`	Todas as páginas abrem sem erro e mostram os textos esperados

Um teste que passa não prova que tudo está certo. Mas um teste que falha avisa antes que o visitante veja o erro. Quando um problema aparece no site, o primeiro passo da correção é escrever o teste que o reproduz.

14 Publicar e manter

O código e os dados moram no GitHub; o site roda no Streamlit Community Cloud, que lê o GitHub. Toda mudança segue o mesmo caminho: arquivos novos no GitHub, o Streamlit percebe e o site se atualiza.

Para subir uma versão nova

1. Descompacte o zip e **abra** a pasta da versão (por exemplo, `portal-guarapuava-v1.2.4`).
2. No GitHub, vá em **Add file → Upload files** e arraste **o que está dentro** da pasta, não a pasta em si.
3. Escolha **Commit directly to the main branch** e confirme.
4. No portal, clique em **Manage app → ⋮ → Reboot app**. Isso faz o Streamlit esquecer as versões antigas dos arquivos que ficaram na memória.
5. Se a versão mudou a coleta, rode o robô uma vez: **Actions → Atualizar dados → Run workflow**.

Arquivos e pastas que começam com ponto (como `.streamlit/config.toml`) costumam ficar escondidos no computador e por isso não vão junto quando você arrasta a pasta. Quando uma versão muda um deles, edite direto no GitHub: abra o arquivo, clique no lápis, cole o conteúdo novo, confirme e faça o Reboot.

O dia a dia

- **Toda segunda-feira** o robô roda sozinho. Quando dá certo, a execução aparece com um sinal verde na aba Actions e surge um commit "Atualiza dados do Tesouro".
- **O site se atualiza sozinho** quando o robô grava dados novos; não precisa de Reboot.
- **Para ver o que o robô fez:** abra a execução na aba Actions, depois o passo "Coletar os dados na API do Tesouro", e procure o "Resumo da coleta" no fim.

Quando algo dá errado

O que aparece	O que quer dizer	O que fazer
`ImportError` no site depois de subir código	O Streamlit ficou com um arquivo antigo na memória	Manage app → ⋮ → Reboot app
Um X vermelho no robô	Uma das APIs falhou ou demorou demais	Abrir o passo "Coletar os dados na API do Tesouro" e ler a mensagem; quase sempre basta rodar de novo
"parcial" na página Sobre os dados	O tempo acabou antes de terminar	Rodar o robô de novo; ele continua de onde parou
Aviso "dados de demonstração"	O portal está lendo dados de teste	Rodar o robô para trazer os dados reais
Uma página com "ainda não coletado"	Aquele módulo ainda não rodou	Rodar o robô; na primeira vez podem ser 2 ou 3 execuções

Desfazer uma mudança

O GitHub guarda todas as versões. Para voltar atrás num arquivo, abra o arquivo, clique em **History**, escolha a versão anterior e copie o conteúdo de volta. Nada se perde de verdade.

PARTE 5 · NA PRÁTICA

15 Estudo de caso: a versão 1.2.4, do pedido à entrega

Esta seção mostra, na ordem em que aconteceu, como o seu último pedido virou código. O caminho é sempre o mesmo: **entender, conferir, decidir, programar, testar, entregar**.

Passo 1: o pedido

1. Usar o **peso no orçamento** como medida principal.
2. Incluir a opção **"só cidades do mesmo porte"**.
3. Manter o **gasto por habitante** como dado secundário, com explicação, e tirar o "salários por habitante".

Passo 2: o diagnóstico, antes de mexer no código

A dúvida era se o gasto por habitante estava certo. A conferência teve três partes:

- **A conta.** Gasto por habitante = despesa liquidada na função, no Balanço Anual (DCA Anexo I-E), ÷ população. O valor de Guarapuava bateu com a soma da MSC: a conta estava certa.
- **O problema.** Mesmo certa, a medida engana. Cidades pequenas gastam mais por habitante, porque têm custos fixos e recebem mais repasses por pessoa. Em educação por habitante (R$ 1.248), Guarapuava ficava em 368º de 388.
- **A alternativa.** O peso no orçamento não depende do tamanho: 26,4% do gasto de Guarapuava vai para a educação, 92º de 388 no estado e 10º entre as 21 cidades de 100 mil a 900 mil habitantes, cuja mediana é 25,8%.

Passo 3: as decisões

- O peso no orçamento vira a medida principal de cada área.
- O gasto por habitante continua, embaixo de cada faixa de ranking, com a posição no mesmo grupo.
- O "salários por habitante" sai: para salários, a medida que a lei usa é o percentual da receita corrente líquida (LRF), que continua na página.
- "Mesmo porte" usa faixas de população conhecidas e fáceis de explicar, e não uma fórmula escondida.

Passo 4: o código, em quatro partes

a) A conta nova (`despesas/comparativo.py`, dentro de `tabela_indicadores`; simplificado):

```
total = base["despesa_total"]                    # tudo o que foi gasto no ano (DCA I-E)
for codigo, nome in FUNCOES.items():             # "12" → "educacao", "10" → "saude"...
    base[f"{nome}_peso"] = base[f"funcao_{codigo}"] / total * 100
base["investimento_peso"] = base["investimentos"] / total * 100
```

O `for` repete a mesma conta para cada área da lista `FUNCOES`. O `f"{nome}_peso"` monta o nome da coluna: `educacao_peso`, `saude_peso` e assim por diante.

b) A lista do que aparece na página (`INDICADORES`). Cada indicador é uma ficha com o nome e, agora, o dado de apoio:

```
{"chave": "educacao_peso", "nome": "Educação: peso no orçamento",
 "curto": "Educação (% do gasto)", "unidade": "%", "tema": "Educação",
 "secundario": "educacao_hab"},          # o dado de apoio: o gasto por habitante
```

Tirar o "salários por habitante" foi apagar a ficha dele da lista. Nenhuma outra parte do código precisou mudar, porque a página monta tudo a partir dessa lista.

c) O grupo do mesmo porte (`cidades_do_mesmo_porte`):

```
faixa = faixa_de_porte(pop[cod])                                  # a faixa da sua cidade
grupo = [c for c, v in pop.items() if faixa_de_porte(v) == faixa]  # todas da mesma faixa
if len(grupo) >= 8:
    # exemplo da frase: "21 cidades de 100 mil a 900 mil habitantes"
    return grupo, f"{len(grupo)} cidades {faixa}"
# faixa com menos de 8 cidades: as 12 de população mais próxima
```

d) A tela (`paginas/comparar.py`; simplificado). O botão de escolha e o recálculo dentro do grupo:

```
c3.radio("Comparar entre", ["Todo o Paraná", "Cidades do mesmo porte"],
         key="cmp_escopo", horizontal=True)

def serie_do_grupo(chave):                       # os valores só das cidades do grupo
    return tab.loc[grupo, chave]

posicao = serie_do_grupo(chave).rank(ascending=False, method="min")   # ranking dentro do grupo
```

Um detalhe importante: o grupo precisa ser calculado **antes** de desenhar os seletores, porque a lista "Comparar com" depende dele. O código lê a escolha anterior em `st.session_state["cmp_escopo"]` logo no começo da página.

Passo 5: os testes

Três testes novos ou ampliados: a conta do peso (`test_peso_no_orcamento`), os grupos de porte, inclusive o caso de Curitiba sozinha na faixa (`test_cidades_do_mesmo_porte`), e a página trocando para "Cidades do mesmo porte" sem erro (`test_comparar_cidades`). Resultado: 92 testes passando, também numa cópia do seu repositório, com os dados reais.

Passo 6: a conferência visual

Com os dados reais, a página foi aberta num navegador automático e fotografada (a foto da página inteira está na seção 9). Foi assim que apareceu um problema que os testes não pegam: a tabela de resumo ficava larga demais e cortava a coluna da mediana. A correção foi definir a largura de cada coluna.



Figura 6. As faixas de ranking na versão 1.2.4: o peso no orçamento em destaque e, embaixo, o gasto por habitante como dado de apoio.

Passo 7: um ajuste extra

Na foto da Visão geral apareceu "1 fornecedores com 3 ou mais dispensas". A correção foi uma função pequena em `despesas/formatacao.py`:

```
def contagem(n, singular, plural):
    return f"{numero(n, 0)} {singular if n == 1 else plural}"  # 1 fornecedor · 2 fornecedores

contagem(1, "fornecedor", "fornecedores")     # → "1 fornecedor"
```

O `singular if n == 1 else plural` é um "se" numa linha só: use o singular se o número for 1; senão, o plural.

Passo 8: a entrega

Um zip só com os 12 arquivos que mudaram, o passo a passo de publicação (seção 14) e o lembrete do Reboot. O robô não precisa rodar: a conta nova usa dados que ele já coletava.

16 Para praticar

Sete exercícios, do mais simples ao mais técnico. Nenhum deles muda o portal publicado: quando houver código, você vai testar num computador na nuvem e desfazer no fim. **Nos exercícios, não faça commit.**

Exercício 1: ache um número na fonte (10 minutos)

1. Copie o endereço da seção 5 (a consulta do RGF), junte tudo numa linha só e cole na barra do navegador.
2. Aperte Ctrl+F e procure por `RCL (IV)`.
3. Confira o valor: 929125228.65.

> **O que você aprende:** uma API é só um endereço, e a resposta em JSON dá para ler com calma.

Exercício 2: os dados do portal no Excel (15 minutos)

1. No portal, abra **Sobre os dados** e clique em **Baixar tudo (ZIP)**.
2. Abra o `fIndicadores.csv` no Excel e ligue o filtro.
3. Filtre: relatorio = RGF, ano = 2026, periodo = 2, poder = E, chave = pessoal_pct.
4. O valor é 51,07. Na Visão geral, é o "51,07% da receita corrente líquida" do ponto de atenção sobre pessoal.

> **O que você aprende:** o portal é a soma destes arquivos com o desenho das páginas.

Exercício 3: leia o diário do robô (10 minutos)

1. No GitHub, abra a aba **Actions** e clique na última execução de **Atualizar dados**.
2. Abra o passo **Coletar os dados na API do Tesouro** e role até o **Resumo da coleta**.
3. Responda: quantos contratos foram coletados? Até que bimestre vai o RREO?

> **O que você aprende:** o log conta tudo o que o programa fez, inclusive os erros.

Exercício 4: rode o robô pelo botão (5 minutos, mais a espera)

1. Em **Actions → Atualizar dados**, clique em **Run workflow**, deixe a caixa "Recoletar tudo" desmarcada e confirme.
2. Acompanhe os passos ficando verdes. Se nada mudou nas fontes, o último passo diz "Nada mudou nos dados."

> **O que você aprende:** o `workflow_dispatch` da agenda é este botão.

Exercício 5: o portal num computador na nuvem (20 minutos)

1. No repositório, clique no botão verde **Code → Codespaces → Create codespace on main**.
2. Espere a preparação. O próprio Codespace abre o portal numa aba de visualização (porta 8501).
3. Abra o `app.py` e mude o texto "Contas públicas da prefeitura em dados abertos e oficiais". Salve com Ctrl+S.

4. Na aba do portal, clique em **Rerun**: o subtítulo muda só ali, não no portal publicado.
5. Desfaça no terminal: `git restore app.py`.
6. No fim, apague o Codespace em github.com/codespaces. O GitHub dá uma cota mensal gratuita, e um Codespace parado ainda ocupa espaço.

> **O que você aprende:** o ciclo de quem programa: mudar, rodar, olhar, desfazer.

Exercício 6: rode os testes (10 minutos)

No terminal do Codespace:

```
pip install -r requirements-dev.txt     # instala o pytest
python -m pytest -q                      # roda os 92 testes
```

No fim deve aparecer `92 passed`.

> **O que você aprende:** os testes conferem o portal inteiro em menos de um minuto.

Exercício 7: quebre um teste de propósito (15 minutos)

1. Em `despesas/alertas.py`, troque `MIN_DISPENSAS_MESMO_FORNECEDOR = 3` por `4` e salve.
2. Rode só os testes dos pontos de atenção: `python -m pytest -q tests/test_alertas.py`.
3. Leia a falha: o pytest mostra a linha do `assert`, o valor esperado e o valor obtido.
4. Desfaça: `git restore despesas/alertas.py` e rode de novo, para ver tudo passar.

> **O que você aprende:** os testes protegem as regras. Uma mudança de critério sempre exige mudar o teste junto, de propósito.

PARTE 5 · NA PRÁTICA

17 Glossário

Termo	O que quer dizer
Altair	Biblioteca de gráficos usada no mapa e nas faixas de ranking
API	Endereço na internet que devolve dados para programas, em vez de páginas para pessoas
`assert`	"Garanta que": a linha de um teste que confere um resultado
Biblioteca	Caixa de ferramentas pronta que o código chama quando precisa (pandas, requests)
Cache	Memória que guarda um resultado para não refazer o trabalho a cada clique
Commit	Um "salvar" no GitHub, com data, autor e descrição; dá para voltar a ele depois
`cron`	Agenda em cinco campos: minuto, hora, dia do mês, mês e dia da semana
CSS	Linguagem que define a aparência de uma página: cores, tamanhos, espaços
CSV	Tabela em texto puro; no portal, colunas separadas por ponto e vírgula
DataFrame	Uma tabela dentro do pandas
DCA	Declaração de Contas Anuais: o balanço do ano que cada município envia ao Tesouro
Dimensão (`d`)	Tabela de nomes que explicam códigos, como `dFuncao` (12 = Educação)
Empenhado, liquidado, pago	As fases da despesa: reservar o dinheiro, confirmar a entrega, pagar
Fato (`f`)	Tabela de valores, como `fDespesa`
Framework	Estrutura pronta que chama o seu código (Streamlit)
Função (programação)	Bloco de código com nome, que recebe dados e devolve um resultado (`def contagem(...)`)
Função de governo	A área do gasto na classificação oficial: 10 Saúde, 12 Educação, 26 Transporte
GitHub Actions	Serviço que roda programas sozinho, numa agenda: o robô
HTTP (código de status)	Resposta do servidor: 200 ok, 204 vazio, 404 não existe, 429 espere, 500 em diante falha do servidor
JSON	Formato de texto para dados, com `{ }`, `[ ]` e `"nome": valor`
LGPD	Lei Geral de Proteção de Dados Pessoais (Lei 13.709/2018)
Log	O diário de uma execução: tudo o que o programa escreveu enquanto rodava
LRF	Lei de Responsabilidade Fiscal (Lei Complementar 101/2000): limites de pessoal e de dívida
MEI	Microempreendedor Individual: empresa de uma pessoa só
Mediana	O valor do meio: metade das cidades fica acima e metade abaixo
MSC	Matriz de Saldos Contábeis: o envio mensal com a contabilidade detalhada
pandas	Biblioteca de tabelas do Python
Parquet	Formato de arquivo compacto para tabelas grandes

Termo	O que quer dizer
Peso no orçamento	De cada R$ 100 gastos no ano, quanto foi para a área
PNCP	Portal Nacional de Contratações Públicas (Lei 14.133/2021)
Por habitante	Valor dividido pela população
pytest	Ferramenta que roda os testes automáticos
Quintil	Cada uma de cinco partes com o mesmo número de cidades
RCL	Receita Corrente Líquida: a receita dos últimos 12 meses usada como base dos limites da LRF
Reboot	Reiniciar o app no Streamlit Cloud para carregar o código novo
Regex	Expressão regular: um molde para procurar textos ("começa com...", "11 dígitos")
Repositório	A pasta do projeto no GitHub, com todo o histórico
RGF	Relatório de Gestão Fiscal, a cada 4 meses: pessoal e dívida
RREO	Relatório Resumido da Execução Orçamentária, a cada 2 meses: receitas, despesas e mínimos
`session_state`	A memória do Streamlit para cada visitante (o ano, a cidade escolhida)
SICONFI	Sistema do Tesouro Nacional que recebe as contas de estados e municípios
Streamlit	Framework que transforma um script Python num site
SVG	Imagem descrita em texto (formas e cores) que fica nítida em qualquer tamanho
Teste automático	Programa que confere se o código faz o que deveria
Token de cor	Um nome para uma cor usada em vários lugares ("superficie", "tinta")
YAML	Formato de configuração em que a indentação mostra o que está dentro de quê